Dashboard 360 de Olist

Tema elegido: 10 · Dashboard 360 del negocio

Integrantes: Favio Jimenez · Roberto Muñoz Velasco

DPL1046 · Programación en Python y Manipulación de Datos

PhD. Juan Manuel Rozas · [fecha]

Ventas, satisfacción y entregas de Olist en un solo panorama ejecutivo

9 tablas · 1 comando · 1 dashboard

El problema de negocio

Una pregunta de gestión sobre nueve tablas que no se pueden sumar sin cuidado

- **Pregunta de negocio:** ¿Cómo se comportan las ventas, las reseñas y las entregas de Olist, y qué categorías y regiones concentran las ventas?

- **¿Para quién?** Para la persona responsable de la gestión comercial u operativa.

- **¿Por qué importa?** Los datos vienen en nueve tablas. Un pedido tiene varios ítems, pagos y reseñas: unirlos sin agregar antes multiplica los importes.

96.478

pedidos entregados analizados (97,02 % de los 99.441 pedidos)

13,2 M

GMV sin flete, en la moneda original de los datos

9

tablas de Olist integradas sin duplicar importes

Los datos

Nueve tablas CSV, 1,55 millones de filas y problemas de calidad reales

Tabla	Formato	Filas	Aporte al dashboard
orders	CSV	99.441	Estado y fechas
order_items	CSV	112.650	Precio, flete y vendedor
order_payments	CSV	103.886	Pagos por pedido
order_reviews	CSV	99.224	Puntaje de satisfacción
products	CSV	32.951	Categoría del producto
category_translation	CSV	71	Traducción de categorías
customers	CSV	99.441	Cliente y estado
sellers	CSV	3.095	Vendedor y ubicación
geolocation	CSV	1.000.163	Coordenadas por prefijo

Total: 1.550.922 filas · 126 MB · los CSV originales no se modifican.

Calidad real de los datos

775 pedidos sin ítems

551 pedidos con más de una reseña

261.831 filas repetidas en geolocalización

2.965 pedidos sin fecha de entrega al cliente

58,7 % de las reseñas sin comentario

Arquitectura del pipeline

Un comando recorre entrada, proceso y salida

ENTRADA

- 9 CSV de Olist en data/raw
- Solo lectura: los originales no se modifican
- config/settings.json: nivel y archivo del log

PROCESO

- etl.py: valida claves, filtra entregados y agrega ítems por pedido
- indicadores.py: reseñas por pedido, entregas y categorías
- contexto.py: clientes, pagos, vendedores y geografía

SALIDA

- SQLite: 8 tablas en data/processed
- Dashboard en Streamlit
- reports/pipeline.log y evidencias de repetición

Herramientas: Python 3.12 · pandas · sqlite3 · Streamlit y Altair · pytest · logging

Carpetas: pipeline.py · src/ · tests/ · config/ · reports/ · dashboard.py

Control: cada unión se valida (uno a uno o muchos a uno) y los totales de pedidos, categorías y vendedores se reconcilian con el GMV total.

Demo en vivo

Un solo comando: python pipeline.py

```
$ python pipeline.py
2026-10-27 10:20:02,531 INFO dashboard360: Inicio del pipeline.
2026-10-27 10:20:02,531 INFO dashboard360: Procesando pedidos e ítems.
2026-10-27 10:20:07,481 INFO dashboard360: Calculando los cinco indicadores.
2026-10-27 10:20:09,013 INFO dashboard360: Integrando clientes, pagos y geolocalización.
2026-10-27 10:20:12,123 INFO dashboard360: Pipeline completado correctamente.
$ python -m streamlit run dashboard.py
```

Qué se ve en vivo

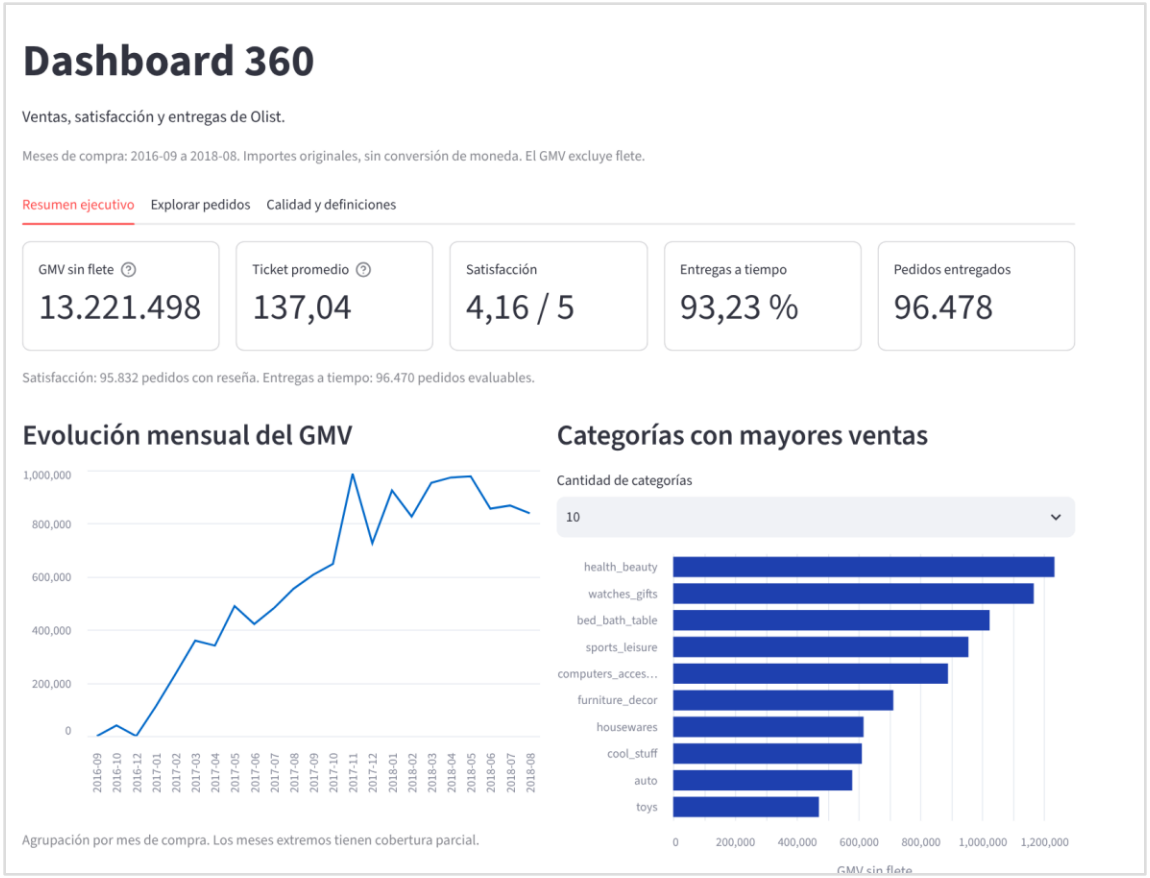
- Etapas en la terminal y cierre «Pipeline completado correctamente»
- Dashboard: resumen ejecutivo, exploración y calidad
- Repetir el comando: mismas 8 tablas y mismos hashes

Respaldo y tiempos

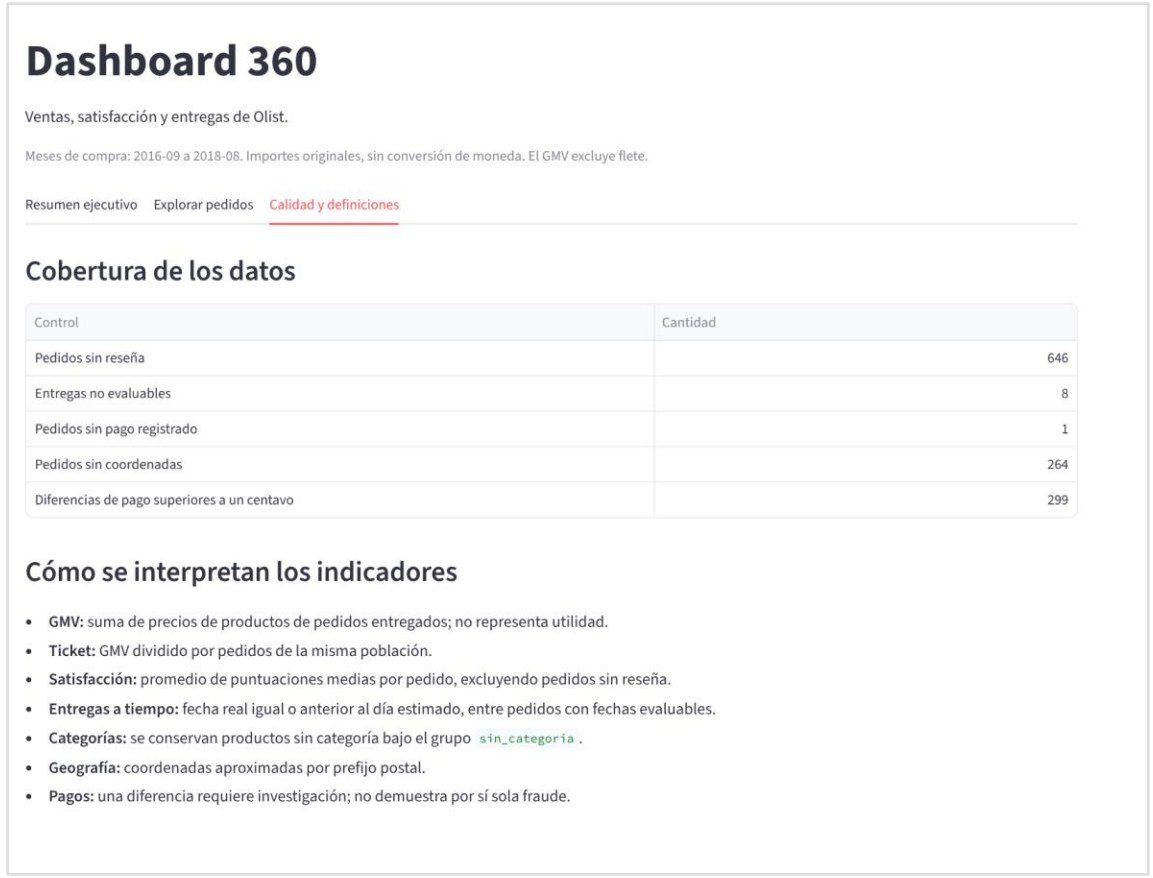
- El pipeline tarda unos 10 segundos (según pipeline.log)
- Plan B: capturas en docs/presentacion/capturas
- Ensayar el día antes, con el computador cargado

Demo en vivo: el dashboard

Los mismos números del pipeline, explorables en el navegador



Resumen ejecutivo: cinco indicadores, evolución mensual y categorías

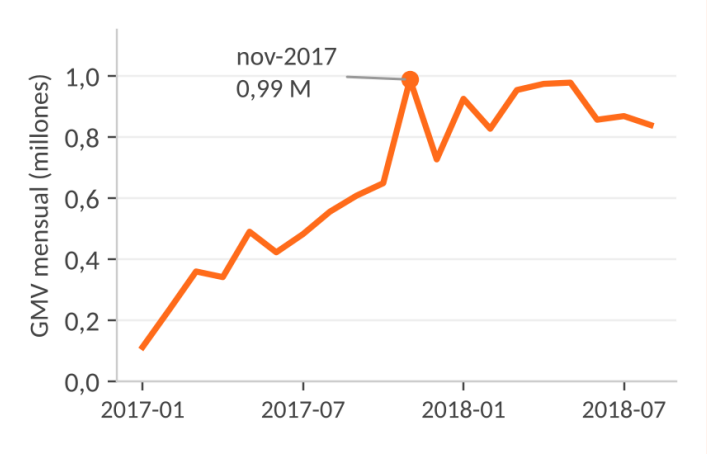


Calidad y definiciones: controles de cobertura y cómo leer cada indicador

Resultados

Ventas estables, categorías concentradas y entregas menos puntuales en Río y Bahía

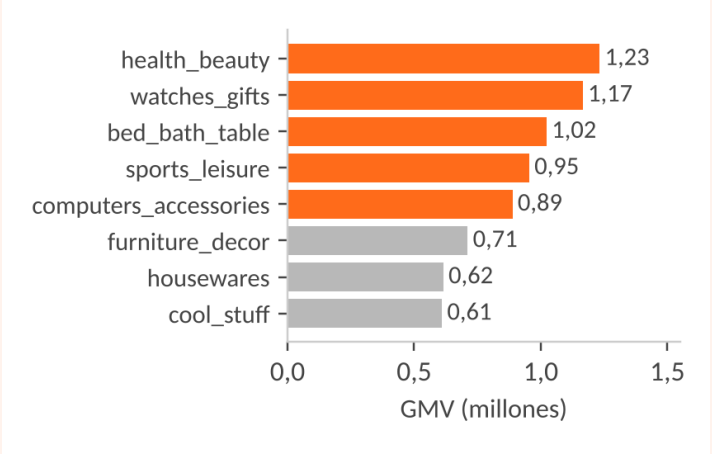
1 · Las ventas se estabilizan



Vemos: El GMV mensual pasó de 0,11 M (ene-2017) a entre 0,83 M y 0,98 M al mes en 2018.

Significa: Hay una base mensual estable; con un solo noviembre en los datos no se puede afirmar estacionalidad.

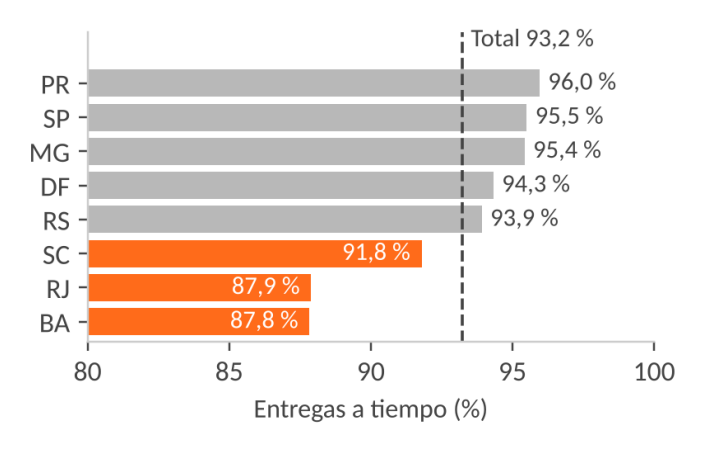
2 · Cinco categorías suman el 39,8 %



Vemos: health_beauty, watches_gifts y otras tres suman 5,27 M de 13,22 M de GMV.

Significa: Son las candidatas para priorizar en surtido y logística.

3 · Río y Bahía entregan menos a tiempo



Vemos: 93,23 % de entregas a tiempo en total; RJ y BA bajan a 87,9 % y 87,8 %, y SP llega a 95,5 %.

Significa: Conviene revisar primero la logística hacia esos estados.

Calidad y buenas prácticas

Calidad verificable: validación, logging, 9 pruebas y repetición sin duplicados

☑ **Validación:** claves nulas o duplicadas, importes negativos, uniones validadas y totales reconciliados

☑ **Errores y logging:** pipeline.log en la terminal y en archivo

☑ **Pruebas pytest:** 9 passed, tres por cada función de transformación de etl.py

☑ **Archivos del proyecto:** requirements.txt, README y config/settings.json

☑ **Buenas prácticas:** rutas relativas, sin credenciales en el código y .env ignorado por Git

☑ **Repetible:** dos ejecuciones, ocho tablas, mismos hashes SHA-256

9 passed

python -m pytest -v

pruebas con tablas pequeñas en memoria

Siguiente paso de calidad

Extraer funciones pequeñas de indicadores.py y contexto.py y probarlas, igual que etl.py.

Valor de negocio

De nueve tablas dispersas a un panorama ejecutivo repetible

Decisión que permite

- Dónde reforzar la logística: RJ y BA entregan a tiempo 87,9 % y 87,8 %.
- Qué categorías priorizar: las cinco principales concentran el 39,8 % del GMV.

Tiempo y esfuerzo

- Un comando reemplaza unir nueve tablas a mano.
- El pipeline completo tarda unos 10 segundos y entrega las mismas cifras en cada ejecución.

Próximos pasos

- Pruebas para indicadores.py y contexto.py.
- Segunda fuente: una API externa (opcional).
- Resumen para gerencia con IA local (opcional).
- Escritura atómica de la base.

Aprendizajes e IA

La IA como copiloto: nosotros escribimos, ejecutamos y comprobamos

Cómo usamos la IA

Codex (VS Code) ayudó a escribir código. Claude ayudó a revisar el sistema, desarrollar la presentación y corregir la documentación. Las instalaciones, las ejecuciones y la verificación de resultados las hicimos nosotros a mano.

Qué corregimos

Errores reales: un NameError por un nombre mal escrito, un ciclo mal indentado que dejaba 7 filas en vez de 15 y ValueError escrito en minúscula. También librerías que faltaban en el entorno y un pandas ya instalado que se mezclaba con el del proyecto; lo resolvimos con un entorno virtual en Python 3.12.

Qué aprendimos

Que se ejecute sin errores no prueba que el resultado sea correcto: se comprueban conteos y totales. Se agrupa antes de unir, para no multiplicar importes. Un entorno virtual evita choques de versiones.

Qué haríamos distinto

Escribir las pruebas desde el inicio, también para los indicadores y la integración de tablas, guardar en SQLite de forma atómica y sumar una segunda fuente de datos.

Preguntas

5 minutos · pueden preguntar por cualquier parte del código